Microservice definition - AuthService
Version: 1.0.0
Description
Backend API with no interface. AuthService will handle the authentication of all user parties : Clients, admins, drivers and server-to-server communication. This service will generate tokens provided the credentials are valid. Other functional needs include refreshing and checking validity of tokens.

Tech stack
- .NET 10
- PostgreSQL
- Redis (later)

Non functional requirements
- Authenticate users
- Authenticate machine-to-machine exchanges
- Manage user accounts
- Manage service accounts

Functional requirements
- Authentication model : OAuth 2.0 M2M, JWKS
- Include role-based claims in tokens
- Generate JWT tokens
- Get current user
- Refresh user tokens
- Blacklist tokens
- Log every action
- CRUD user accounts

Architecture Overview
The application follows a layered architecture:
- Controllers: HTTP API endpoints
- Services: Business logic and workflow orchestration
- Models: Core entities (Invoice, Client, Product, Company, etc.)
- Data: Entity Framework Core with PostgreSQL
- DTOs : Data Transfer Objects to exchange data
- Models : Data models for exchanges between EF and PostgresSQL
- Enums:

```
AuthService/
├── Controllers/        # HTTP API endpoints
├── Services/           # Business logic and workflow orchestration
├── Core/               # Core domain entities & business rules (Invoice, Client, Product, Company, etc.)
├── Data/               # Entity Framework Core DbContext and configurations
├── DTOs/               # Data Transfer Objects for API requests and responses
├── Models/             # Data models / Database entities for exchanges between EF and PostgreSQL
└── Enums/              # System enums (Roles, Statuses, etc.)
```

Key Class Definitions & Responsibilities

1. **Controllers/AuthController.cs**
    - Acts as the entry point handling HTTP requests. It depends *only* on application services, preserving the separation of concerns.
2. **Services/AuthService.cs & TokenService.cs**
    - Encapsulates core business logic (credential verification, claims mapping, password hashing, and cryptographic JWT/Refresh token generation).
3. **Core/User.cs**
    - The domain model encapsulating business state and domain validation rules (e.g., checking if an account is active or updating passwords safely).
4. **Models/UserEntity.cs**
    - The persistence database model mapped strictly to your PostgreSQL tables via Entity Framework Core, keeping database schema concerns out of the core domain layer.
5. **DTOs/**
    - Input/output data structures (LoginDto, TokenResponseDto) ensuring that external API clients never directly bind to database or domain models.
6. **Enums/UserRole.cs**
    - Defines system roles (ADMIN, DRIVER, CLIENT, M2M) to feed into token claims and role-based authorization policies across microservices.

Data Model

```sql
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    email VARCHAR(255) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
```

```
    role VARCHAR(50) NOT NULL CHECK (role IN ('ADMIN', 'DRIVER',
'CLIENT')),
    is_active BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_users_email ON users(email);



--
================================================================================
======
-- USER PROFILES & ACTORS (ShipmentService)
--
================================================================================
======

-- Administrator profile linked to core user account
CREATE TABLE admins (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID UNIQUE NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    phone_number VARCHAR(50),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Client / Customer profile (shipment senders and receivers)
CREATE TABLE clients (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    user_id UUID UNIQUE NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    company_name VARCHAR(255),
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    phone_number VARCHAR(50) NOT NULL,
    address TEXT NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

-- Delivery Driver profile
CREATE TABLE drivers (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
    user_id UUID UNIQUE NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    first_name VARCHAR(100) NOT NULL,
    last_name VARCHAR(100) NOT NULL,
    license_number VARCHAR(100) UNIQUE NOT NULL,
    phone_number VARCHAR(50) NOT NULL,
    is_available BOOLEAN NOT NULL DEFAULT TRUE,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);
```

API Design

Endpoint	Method	Description	Access Level
/api/v1/auth/register	POST	Register a new user (Client/Driver)	Public
/api/v1/auth/login	POST	Authenticate user & return Access/Refresh tokens	Public
/api/v1/auth/refresh	POST	Exchange a refresh token for a new access token	Public (with token)
/api/v1/auth/logout	POST	Revoke/blacklist the current refresh token	Authenticated
/api/v1/auth/m2m/token	POST	OAuth2 Client Credentials grant for M2M services	Service Credentials

/api/v1/auth/jwks.json	**GET**	**Public JWKS endpoint for token verification**	**Public**
/api/v1/users/me	**GET**	**Get profile details of the currently logged-in user**	**Authenticated**
/api/v1/users/{id}	**PUT/DELETE**	**Admin/Self management of accounts**	**Admin / Owner**

Token Claims

1. User Access Token Spec & Claims Table

Used when a human user (Client, Driver, or Admin) authenticates via credentials and calls downstream APIs.

- Grant Type: Password / Authorization Code
- Typical Lifespan: 15 Minutes

Claim Name	Claim Type	Description / Value Example
iss	**Standard (RFC 7519)**	**Issuer URL of the AuthService (e.g., "https://auth.lastmile.internal")**
sub	**Standard (RFC 7519)**	**Unique user account ID (UUID from PostgreSQL users table, e.g., "f81d4fae-7dec-11d0-a765-00a0c91e6bf6")**
aud	**Standard (RFC 7519)**	**Intended target microservice or API gateway (e.g., "shipment-service")**

exp	**Standard (RFC 7519)**	**Expiration Unix epoch timestamp (e.g., 1773014400)**
iat	**Standard (RFC 7519)**	**Issued-at Unix epoch timestamp (e.g., 1773013500)**
jti	**Standard (RFC 7519)**	**Unique JWT ID for blacklisting/revocation tracking (UUID string)**
role	**Custom Profile**	**User permission level matching database check constraints ("ADMIN", "DRIVER", or "CLIENT")**
email	**Custom Profile**	**User contact email for identification (e.g., "driver.jane@lastmile.com")**
profile_id	**Custom Profile**	**Foreign key reference to profile table (admins, clients, or drivers table ID)**

2. Machine-to-Machine (M2M) Token Spec & Claims Table

Used via the OAuth 2.0 Client Credentials Grant when backend services communicate securely without a human user.

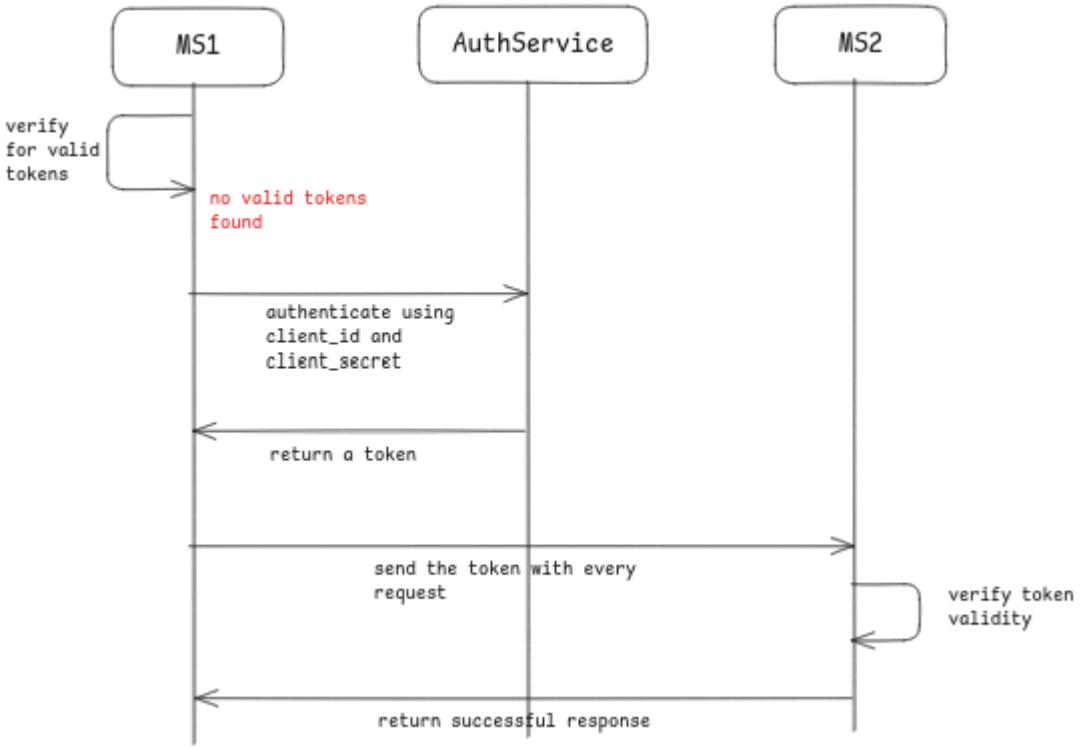
- Grant Type: client_credentials
- Typical Lifespan: 1 Hour (up to 4 Hours)

Claim Name	Claim Type	Description / Value Example

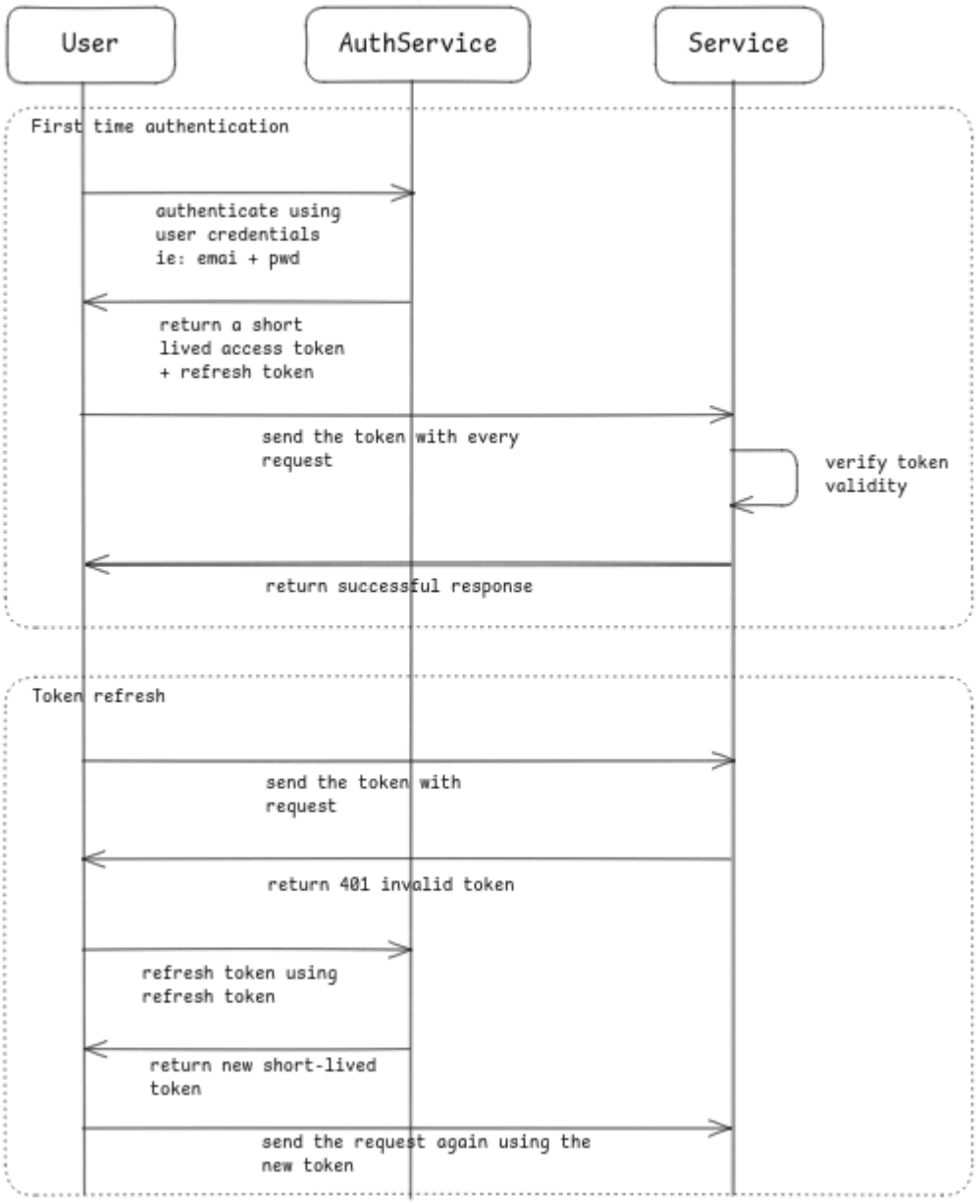
iss	**Standard (RFC 7519)**	**Issuer URL of the AuthService (e.g., "https://auth.lastmile.internal")**
sub	**Standard (RFC 7519)**	**Client identifier representing the service principal (e.g., "shipment-service@clients")**
aud	**Standard (RFC 7519)**	**Downstream resource server or API being accessed (e.g., "inventory-service")**
exp	**Standard (RFC 7519)**	**Expiration Unix epoch timestamp (e.g., 1773018000)**
iat	**Standard (RFC 7519)**	**Issued-at Unix epoch timestamp (e.g., 1773014400)**
jti	**Standard (RFC 7519)**	**Unique JWT ID for tracking/audit logging (UUID string)**
client_id	**OAuth 2.0 Spec**	**Unique system identifier of the calling service script/daemon (e.g., "shipment-service-worker")**
scope	**OAuth 2.0 Spec**	**Space-delimited permissions granted to the service (e.g., "packages:read inventory:write")**
gty	**OAuth 2.0 Spec**	**Explicit grant type indicator confirming M2M execution ("client_credentials")**

Authentication Workflows

1. M2M



2. User Atuh


User
AuthService
Service
First time authentication
authenticate using
user credentials
ie: emai + pwd
return a short
lived access token
+ refresh token
send the token with every
request
verify token
validity
return successful response
Token refresh
send the token with
request
return 401 invalid token
refresh token using
refresh token
return new short-lived
token
send the request again using the
new token